CHAI-FP: Characterizing Hierarchical AI Model Fingerprints from Network Traffic Metadata

Author Names
Affiliation
City, Country
email@domain

Abstract—Federated learning (FL) keeps training data on client devices, but the network traffic it generates can still reveal which model is being trained. We present CHAI-FP, a framework that characterizes how much AI model identity a passive observer can infer from packet-level metadata alone, without inspecting payloads or model parameters. CHAI-FP reconstructs the round structure of FL traffic and identifies models hierarchically at the family, architecture, and variant levels. Its key element is cross-device fingerprinting. Because FL clients train the same model on heterogeneous hardware and operating systems, the differences between their traffic reveal the architecture's compute profile. On a physical testbed with three heterogeneous clients (two Ubuntu, one Windows) and 32 experiments spanning convolutional, recurrent, transformer, and autoencoder models, cross-device fingerprinting identifies the family of unseen models with a balanced accuracy of 0.74, compared with 0.58 for per-client classification, and raises family accuracy in 5 s windows from 0.31 to 0.51. Within a family, the architecture is identified with a mean balanced accuracy of 0.79 from complete traces, and convolutional architectures with 0.71 from 5 s windows. The model payload size is a baseline leak that reveals the exact variant in every trace, and in 69% from a single model download within a 5 s window, but it is removed by padding, whereas family identification from timing alone remains at 0.62, showing that concealing model identity in FL requires timing defenses in addition to size padding.

Index Terms—Federated learning, traffic analysis, side channels, model fingerprinting, network security, privacy.

I. Introduction

Artificial intelligence (AI) workloads are increasingly executed across edge devices, cloud services, and distributed learning systems [1], [2]. Although encryption and access controls protect transmitted content and model artifacts, they do not conceal all characteristics of the communication generated during execution: packet sizes, directions, timing, volumes, and burst patterns remain observable to a network intermediary [3]. Such metadata may reveal properties of an AI workload even when the observer cannot inspect model parameters, application data, or payloads [4]. Federated learning (FL) is particularly exposed, because every training round produces a characteristic exchange in which each client downloads the global model, trains locally, and uploads an update.

Existing Research: Prior studies infer neural network information from indirect execution observations, including memory accesses, shared caches, GPU execution traces, and system resource measurements [5]–[8]. These observations require physical access to, or co-location with, the executing device. Network traffic is a more remote observation surface, on which communication protocols, training schedules, transport behavior, and network conditions may obscure architectural differences. Encrypted-traffic analysis reveals visited websites [3], [9]–[11], and PACKETPRINT [12] identifies mobile applications and user activities from encrypted wireless packet sequences, although its hierarchy captures traffic patterns rather than structural relationships among AI models. For AI workloads, network traffic has been used to discriminate CNN and RNN families in FL [4], to classify CNN and RNN architectures and variants from encrypted wireless traffic with results reported primarily at the family level [13], to identify AI applications and their execution phases [14], and to identify AI families from 5G scheduling metadata [15].

Gap: The extent to which passive network observations reveal the structural identity of AI models beyond the family level remains insufficiently characterized. It is not established whether an observer can progressively distinguish families, architectures, and variants, how much of this information is available within short observation windows, or whether it survives countermeasures such as size padding. Moreover, existing approaches classify each client's traffic independently and do not exploit a defining property of FL: all clients train the same model concurrently on heterogeneous hardware and operating systems.

Motivation: Three technical challenges motivate this investigation. First, different families and architectures may generate similar communication patterns under common FL protocols, so network similarity is an imperfect indicator of model structure. Second, discriminative characteristics of complete traces may be unavailable within short windows, which capture only portions of a round. Third, observations from the same execution are correlated, so randomly distributing them between training and testing produces optimistic estimates; generalization must instead be assessed on models absent from training. Architectural leakage also has security consequences. Knowledge of a model's family, architecture, or variant narrows the range of potential targets and helps an adversary select relevant

attack techniques, for instance by training a surrogate of the matching architecture for model extraction or transfer-based adversarial attacks [16], and it exposes proprietary design choices.

This Work: We present CHAI-FP, a conditional hierarchical framework that infers the family, architecture, and variant of the model under training from passive packet metadata. CHAI-FP reconstructs the round structure of FL traffic, from which it derives communication features and a passive estimate of local training time, the compute gap between model download and upload, and it combines the traffic of all clients into a cross-device representation. We evaluate it on a physical FL testbed with three clients that differ in both hardware and operating system, and 32 experiments, using model-disjoint validation on 0.5 to 5 s windows and complete traces.

Contributions: The contributions of this study are threefold. (i) A conditional hierarchical fingerprinting framework that identifies model families, architectures, and variants from passive network metadata, combining packet-morphology features with compute-timing features derived from the FL round structure; within a family, packet-morphology features identify the architecture with a mean balanced accuracy of 0.79. (ii) Cross-device fingerprinting, which exploits the heterogeneous clients of a federated job to raise family identification of unseen models from 0.58 to 0.74 for complete traces and from 0.31 to 0.51 for 5 s windows, where it also identifies convolutional architectures with 0.71; an ablation shows that the gain requires both the cross-device representation and a regularized linear classifier. (iii) A model-disjoint evaluation of the observable leakage, showing that payload size is a baseline leak that reveals the exact variant in every trace, and in 69% from a single model download within a 5 s window, but is removed by padding, whereas family identification from timing alone remains at 0.62, so that concealing model identity requires timing defenses.

The remainder of the paper is organized as follows. Section II describes the experimental setup, Section III presents the CHAI-FP framework, Section IV reports the results, Section V discusses countermeasures, and Section VI concludes the paper.

II. Experimental Setup

The experimental testbed consists of three AI execution clients, a coordinating server, and a passive network proxy positioned along the communication path. The testbed is heterogeneous in both hardware and operating system: the clients are an NVIDIA Jetson Orin Nano and a Dell desktop running Ubuntu, and a Dell laptop running Windows 11, whereas the server, hosted on an NVIDIA Jetson AGX Orin, and the proxy run Ubuntu. The clients therefore differ in compute capability as well as in their network stacks; for example, the Windows client omits the TCP timestamp option that the Ubuntu hosts use by default. In two experiments, the Jetson AGX Orin also served as the embedded client. Clients connect to the proxy, which relays their traffic to the server, so every client–server exchange traverses the observation point. The proxy captures packet-level metadata, including packet sizes, directions, timing, traffic volumes, and burst characteristics, without accessing model parameters or application payloads. These quantities are observable whether or not the payload is encrypted. Table I summarizes the AI workloads, architectural configurations, and training datasets. Each experiment executes 100 federated learning rounds. Federated training uses a custom synchronous FedAvg implementation in PyTorch, in which the server and clients exchange length-prefixed messages over TCP; in each round, every client downloads the global model, trains locally, and uploads one update before the server aggregates and advances to the next round. The experimental corpus comprises 32 independent experiments and 96 client-level aggregated observations. Traffic is represented using 0.5, 1, 2, and 5 s observation windows and complete-trace aggregates to examine fingerprinting performance across temporal scales. Classification is evaluated across the family, architecture, and variant levels of the model hierarchy, with the validation protocol detailed in Sec. III-G.

TABLE I
AI Workloads, Architectures, Variants, and Datasets

Family	Architecture	Representative Variants	Dataset
Autoencoder	Dense, Conv., VAE	Layer configurations, β-VAE, Conv. VAE	Fashion-MNIST
CNN	ResNet, MobileNet, EfficientNet	ResNet18/34/50/101, MobileNetV2/V3, EfficientNet-B0/B1/B2	Fashion-MNIST
RNN	LSTM, GRU	Single-layer, multi-layer, bidirectional variants	UCI HAR
Transformer	BERT, Distil-BERT, Tiny Transformer	BERT model sizes, lightweight configurations	AG News

III. Methodology

The proposed hierarchical AI fingerprinting framework identifies AI model characteristics from network side-channel observations without requiring access to application payloads or model parameters. As illustrated in Fig. 1, the framework consists of passive traffic observation, multiscale feature extraction, hierarchical classification, and fingerprint evaluation.

A. Fingerprinting Dataset and Ground Truth

The fingerprinting dataset is derived from the proxy captures of all experiments and comprises approximately 499 million packets (670 GB, 190 hours of traffic; median trace duration 53 min). Table II summarizes its composition: four families, 11 architectures, and 32 variants, each realized by one experiment observed from three clients. Each client trace yields one complete-trace observation

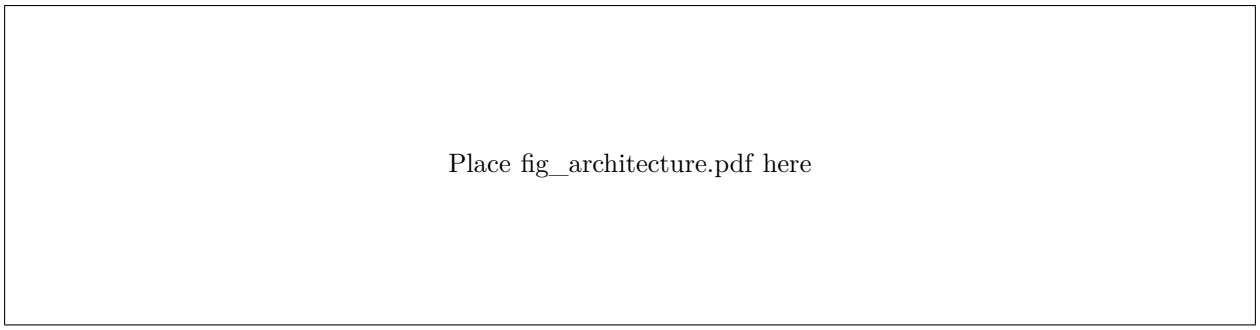


Fig. 1. Overview of the CHAI-FP hierarchical AI fingerprinting framework.

and a sequence of windows (Sec. III-B). Round-level quantities (payload size and compute gap) are available for the 90 traces of the 30 experiments whose rounds are fully reconstructed. Window counts are imbalanced across families, because larger models produce longer traces; training therefore weights every experiment equally, and performance is reported as balanced accuracy.

Each observation is described by 68 size-normalized traffic features in seven groups: direction and volume ratios (7), transmission rates (6), packet-size statistics (20), interarrival-time statistics (12), burst statistics (17), idle time (2), and TLS record sizes (4); the latter are constant in this plaintext testbed and are discarded by feature selection. Complete-trace observations additionally carry nine compute-timing features derived from the round structure: the compute gap per transmitted megabyte, its dispersion and spread, the ratios of round period to compute gap and of waiting time to round period, and cross-device compute-gap and throughput ratios. This yields 77 features per client and 231 in the cross-device representation (9).

TABLE II
Fingerprinting Dataset Composition

Family	Arch.	Var.	Exp.	Traces	5 s win.	0.5 s win.
Autoencoder	3	9	9	27	7,943	57,786
CNN	3	9	9	27	35,947	298,965
RNN	2	6	6	18	2,578	12,036
Transformer	3	8	8	24	39,585	345,817
Total	11	32	32	96	86,053	714,604

Windows with at least 20 packets. The 1 s and 2 s representations contain 381,364 and 201,174 windows; the cross-device representation contains one vector per experiment (32) and 45,559 time-aligned 5 s windows.

Ground-truth labels at each level of the hierarchy, together with per-model metadata (parameter count, serialized size, and training FLOPs per sample), are computed directly from the model source code rather than inferred from traffic, and are used solely for evaluation; the classifiers never observe them. Training FLOPs are obtained from an operator-level cost model, with an analytic correction for recurrent layers whose fused kernels are otherwise uncounted. These labels define the supervision and the hierarchy $\mathcal{H}$ of (5), and the FLOP estimates support the analysis of architectures whose compute profiles yield similar communication patterns.

B. Data Preprocessing

Raw captures are converted into analysis-ready traces in five steps.

Trace assembly. For every client connection, the proxy records a sequence of packet entries (timestamp, frame length, direction, and TCP flags). Long experiments are captured in consecutive chunks, which are placed on a common timeline using their recorded capture-start offsets, and each trace is mapped to its client device through the client address logged by the proxy. Connection attempts that complete no training round are discarded.

Payload recovery. The TCP payload of each packet is obtained by subtracting the Ethernet, IPv4, and TCP header length from the frame length. The header length is inferred per connection from the size of pure acknowledgments: 66 bytes when the TCP timestamp option is used and 54 bytes when it is not, as on the Windows client. Packets without payload, such as pure acknowledgments, are excluded from transfer reconstruction.

Transfer and round reconstruction. Transfers are recovered as maximal runs of same-direction data packets, tolerating small opposite-direction control packets (at most 300 B) and transient stalls caused by wireless retransmission, which must not split a transfer. Because rounds alternate strictly between download and upload, the model-carrying transfers are identified as the byte-weighted dominant size class in each direction, and each upload is paired with the preceding download to delimit a round.

Windowing and filtering. Each trace is segmented into non-overlapping windows on the global timeline (Sec. III-D). Windows with fewer than 20 packets are discarded, and near-idle windows are additionally removed for cross-device window classification.

Normalization. Features that scale directly with model size or trace length (absolute byte and packet counts, durations, and connection and TCP flag counts) are excluded, so that the remaining features describe the shape of the traffic rather than its volume. Missing and non-finite values, which arise for windows without traffic

in one direction or for an absent device in the cross-device representation, are set to zero for windows and to the feature median for traces. Features are standardized using statistics computed on the training folds only.

C. Traffic Observation and Feature Extraction

Network traffic exchanged between AI clients and the coordinating server is observed at an intermediate proxy. Each captured packet is represented by observable metadata, including its timestamp, size, and communication direction. Let

$$p_i = (t_i, s_i, d_i) \tag{1}$$

denote the ith observed packet, where t_i is its arrival timestamp, s_i is its size, and d_i indicates its communication direction. From the resulting packet sequence, the framework extracts statistical features describing packet sizes, interarrival times, directional traffic volumes, transmission rates, and burst behavior.

Beyond per-packet statistics, we exploit the round structure of federated training. Each round consists of a model download, a local-training interval with no traffic, and a model upload. From the directional packet sequence we recover, for round k, the end of the download t_k^{d} and the start of the upload t_k^{u}, and define the compute gap

$$g_k = t_k^{\mathrm{u}} - t_k^{\mathrm{d}}, \tag{2}$$

a passive proxy for local training time that excludes the inter-client synchronization wait. Together, these measurements form a network fingerprint representing the communication and computation characteristics of the executing AI workload. The per-trace summary augments the packet-morphology features with the compute gap normalized by uploaded payload, its dispersion, and cross-device timing ratios (Sec. III-F).

D. Multiscale Traffic Representation

To investigate the temporal availability of AI fingerprints, the observed traffic is represented at multiple time scales. For an observation duration Δ, packets are grouped into temporal windows, and a feature vector is computed for each window:

$$\mathbf{x}_k^{(\Delta)} = \phi\left(\{p_i \mid k\Delta \le t_i < (k+1)\Delta\}\right), \tag{3}$$

where $\phi(\cdot)$ denotes the traffic feature extraction function. The framework considers observation durations $\Delta \in \{0.5, 1, 2, 5\}$ s. In addition, complete client traces are summarized into aggregated fingerprints to characterize communication behavior over the entire observed execution.

A single short window observes only a fragment of one round. We therefore aggregate a client's window-level posteriors to a trace-level decision by averaging,

$$P(y \mid \mathcal{T}) = \frac{1}{|\mathcal{T}|} \sum_{k \in \mathcal{T}} P\Big(y \mid \mathbf{x}_k^{(\Delta)}\Big), \tag{4}$$

where $\mathcal{T}$ indexes the windows of a trace. This recovers the round-level signal lost to short windows without additional training, and enables comparison between short-duration traffic signatures and complete-trace observations.

For cross-device window classification, the three clients' windows covering the same global interval are concatenated, analogously to (9). Near-idle windows, whose transferred bytes fall below 5% of the trace's busy-window volume or whose retransmission share exceeds 5%, are removed before classification; this filter is fixed in advance and uses no labels. For short windows, variant identity is assessed by nearest-reference matching on the size of a model download that completes within the window. To study observation times beyond the window length, the posteriors of consecutive 5 s windows are averaged with (4) over non-overlapping spans of 10 to 30 s.

E. Hierarchical AI Model Fingerprinting

Rather than treating all AI models as unrelated classes, the proposed framework represents model identity through a three-level hierarchy:

$$\mathcal{H} = \mathcal{F} \to \mathcal{A} \to \mathcal{V}, \tag{5}$$

where $\mathcal{F}$, $\mathcal{A}$, and $\mathcal{V}$ denote model families, architectures, and variants, respectively. Given a network fingerprint $\mathbf{x}$, family identification is performed as

$$\hat{f} = \arg\max_{f \in \mathcal{F}} P(f \mid \mathbf{x}). \tag{6}$$

Architecture identification is then conditioned on the selected family:

$$\hat{a} = \arg\max_{a \in \mathcal{A}(\hat{f})} P(a \mid \mathbf{x}, \hat{f}), \tag{7}$$

where $\mathcal{A}(\hat{f})$ denotes the architectures associated with the selected family. Finally, variant identification is conditioned on the selected architecture:

$$\hat{v} = \arg\max_{v \in \mathcal{V}(\hat{a})} P(v \mid \mathbf{x}, \hat{a}), \tag{8}$$

where $\mathcal{V}(\hat{a})$ represents the candidate variants belonging to the selected architecture. This hierarchical formulation preserves the structural relationships among AI models and enables fingerprinting at progressively finer levels of model identity. To isolate per-level discriminability, each level is also evaluated independently using the ground-truth parent.

F. Cross-Device Fingerprinting

In federated learning, the participating clients train the same model on heterogeneous hardware and operating systems. Classifying each client in isolation discards the most discriminative cue available to a passive observer: how the timing and transfer shape of the identical model differ across devices, which is governed by the architecture's per-layer compute profile. We therefore represent each

experiment e by a single feature vector that concatenates the per-device summaries,

$$\mathbf{x}_e = \left[\, \mathbf{f}_e^{(1)} \,\|\, \mathbf{f}_e^{(2)} \,\|\, \mathbf{f}_e^{(3)} \,\|\, \mathbf{r}_e \,\right], \tag{9}$$

where $\mathbf{f}_e^{(c)}$ is the size-normalized feature vector of client c and $\mathbf{r}_e$ collects cross-device compute-gap and throughput ratios. Because (9) yields few high-dimensional examples, family identification (6) is performed with an ℓ_2-regularized multinomial logistic regression whose regularization strength is chosen by nested cross-validation (Sec. III-G); tree ensembles overfit this regime.

G. Feature Selection, Classification, and Validation

The extracted traffic fingerprints train supervised classifiers for each hierarchical identification task. Feature discriminability is examined using Fisher-score-based selection, which evaluates the separation between classes relative to their within-class variability, and is performed independently within each training fold to prevent information leakage from held-out executions. For the per-client representations we use Fisher-selected random forests; for the cross-device representation (9) we use regularized logistic regression. Exact-model identification is performed by nearest-reference matching on the logarithm of the uploaded payload size.

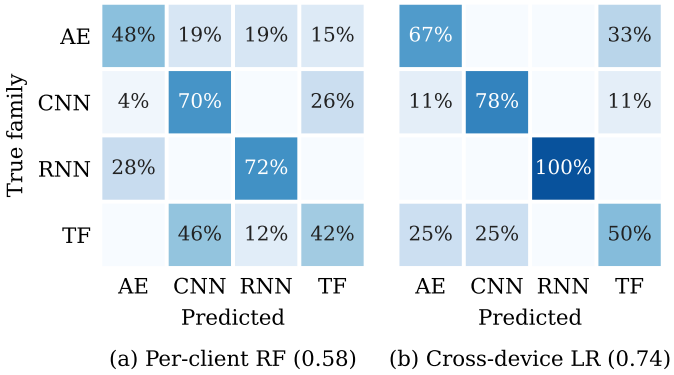
Generalization is assessed under experiment-disjoint, leave-one-model-out validation, in which every held-out model is absent from training, so performance measures recognition of unseen models rather than memorization. All hyperparameter and feature-selection choices are made within an inner loop over the training models, so held-out executions never influence model selection. Performance is reported as balanced accuracy, whose chance level is $1/|\mathcal{C}|$ for a $|\mathcal{C}|$-class task, independent of class sizes; reliability is quantified by a group-bootstrap 95% confidence interval over models and a label-permutation test against chance.

At the variant level, each model is realized by a single execution, and variant identity is recovered through exact-model identification: every variant produces a distinct serialized size, enabling nearest-reference matching on the uploaded payload. Confusion matrices and feature discriminability are examined to characterize residual ambiguity among AI workloads, particularly where different architectures exhibit similar network communication patterns.

Robustness to size concealment is assessed by simulating a padding defense in which every transfer is padded up to a size bucket, and by repeating family identification using only features independent of transfer size: the compute gap, its dispersion, and cross-device gap ratios.

IV. Results

We report fingerprinting performance at the family, architecture, and variant levels, then for short observation windows and under a size-padding defense. Scores are balanced accuracy, for which chance is $1/|\mathcal{C}|$.



(a) Per-client RF (0.58) (b) Cross-device LR (0.74)

Fig. 2. Family confusion matrices for (a) the per-client baseline and (b) the cross-device representation (leave-one-model-out). Cells give the percentage of each true family assigned to each predicted family; empty cells are zero. Balanced accuracy in parentheses.

A. Family Identification

Table III compares family identification methods. Under the repeated protocol used for the earlier methods (20 seeds × 5-fold model-disjoint cross-validation), the cross-device representation of (9) achieves 0.68 ± 0.04, the highest among all methods evaluated this way (fixed late fusion: 0.61) and 0.20 above a per-client random forest (0.48 ± 0.04). Under leave-one-model-out validation, which trains on all remaining models, it reaches 0.74 (95% CI 0.58–0.88, macro F1 0.74), significantly above chance (permutation test, $p = 0.003$), compared with 0.58 for the per-client random forest. Recurrent networks are identified without error (6/6), convolutional networks in 7 of 9 cases, autoencoders in 6 of 9, and transformers in 4 of 8 (Fig. 2). The remaining errors involve architectures whose compute per parameter overlaps.

TABLE III
Family Identification (4 classes, chance 0.25)

Method	Balanced accuracy	
	Repeated CV	Leave-one-model-out
306-feature morphology (RF)	0.55	–
Compute-per-byte	0.57	–
Fixed late fusion	0.61	–
Equal-probability hybrid fusion	0.68*	–
Per-client RF	0.48	0.58
Cross-device LR (proposed)	0.68	0.74

Repeated CV: mean over 20 seeds × 5-fold model-disjoint cross-validation. Leave-one-model-out: nested selection, 95% CI 0.58–0.88 for the proposed method. *Single 5-fold run.

B. Architecture Identification

Using packet-morphology features, the architecture is identified within each family well above chance (Table IV). Recurrent architectures are separated perfectly (1.00), autoencoder architectures reach 0.89, and convolutional architectures 0.78. Transformer architectures are the most difficult (0.47), partly because DistilBERT is represented by a single model. The mean across families is 0.79

(macro F1 0.78), and the cross-device representation raises convolutional architectures further to 0.81.

TABLE IV
Architecture Identification Within Each Family (Balanced Accuracy)

Method	AE	CNN	RNN	Transf.	Mean
Per-client RF (packet)	0.89	0.78	1.00	0.47	0.79
Cross-device LR	0.67	0.81	1.00	0.25	0.68
Chance	0.33	0.33	0.50	0.33	–

C. Variant Identification

At the finest level of the hierarchy, the uploaded payload size provides a baseline leak. It is recovered from traffic within a median relative error of 0.025% of the true serialized size. Because every variant has a distinct size, nearest-reference matching identifies the exact variant in all 90 traces (chance 3.4%). Tables V–VIII report the separability of the variants within each family. Accounting for the client-specific header length is essential: assuming a single header length under-counts the Windows client's payload by 0.8% and causes three of the 90 identifications to fail. Within a 5 s window, every variant with a short download is identified, including all recurrent networks, autoencoders, MobileNets, and Tiny Transformers, whereas EfficientNet, ResNet, and larger BERT downloads exceed the window.

TABLE V
Variant Identification: Autoencoder

Arch.	Variant	MB	Trace	5 s
Conv. AE	2-layer	0.01	1.00	1.00
	4-layer	0.16	1.00	1.00
	6-layer	0.75	1.00	1.00
Dense AE	2-layer	6.31	1.00	1.00
	3-layer	8.14	1.00	1.00
	5-layer	10.11	1.00	0.67
VAE	β/Conv. VAE*	3.44	1.00	1.00
	FC VAE	6.51	1.00	1.00
Autoencoder overall			1.00	0.96

Accuracy of nearest-reference matching on log payload size (leave-one-trace-out; three traces per variant, one per client device). Trace: complete trace; 5 s: one complete download within a 5 s window that begins at the download. *Identical network, two experiments.

TABLE VI
Variant Identification: CNN

Arch.	Variant	MB	Trace	5 s
EfficientNet	B0	16.34	1.00	0.00
	B1	26.48	1.00	0.00
	B2	31.26	1.00	0.00
MobileNet	V3-Small	6.22	1.00	1.00
	V2	9.16	1.00	1.00
ResNet	18	44.80	1.00	0.00
	34†	–	n/a	n/a
	50	94.41	1.00	0.00
	101	170.66	1.00	0.00
CNN overall			1.00	0.25

†Incomplete proxy capture; rounds not reconstructed.

TABLE VII
Variant Identification: RNN

Arch.	Variant	MB	Trace	5 s
GRU	1-layer	0.22	1.00	1.00
	2-layer	0.62	1.00	1.00
	BiGRU, 2-layer	1.62	1.00	1.00
LSTM	1-layer	0.29	1.00	1.00
	2-layer	0.82	1.00	1.00
	BiLSTM, 2-layer	2.16	1.00	1.00
RNN overall			1.00	1.00

TABLE VIII
Variant Identification: Transformer

Arch.	Variant	MB	Trace	5 s
BERT	Tiny	6.85	1.00	1.00
	Mini	23.30	1.00	0.00
	Small	72.27	1.00	0.00
	Base	373.78	1.00	0.00
DistilBERT	Base†	–	n/a	n/a
Tiny Transf.	2-layer	6.78	1.00	1.00
	4-layer	8.37	1.00	1.00
	6-layer	9.97	1.00	1.00
Transformer overall			1.00	0.57

†Incomplete proxy capture; rounds not reconstructed.

D. Discriminative Features

Table IX lists the features with the highest Fisher scores for each task, computed on all observations for interpretation; during evaluation, features are selected within each training fold. Transmission rates separate families in per-client traces, whereas the cross-device representation relies on burst and packet-size dispersion, led by the laptop's view of the same model. The compute-timing features rank first for architecture identification in three of the four families, through the compute gap per megabyte for autoencoders and the cross-device gap ratio for recurrent networks and transformers.

TABLE IX
Top Fisher-Score Features per Task

Task	Top-3 features (Fisher score)
Family, per-client	Download byte rate (0.91), byte rate (0.90), upload byte rate (0.90)
Family, cross-device	Laptop burst SD in packets (1.89), laptop burst SD in bytes (1.87), laptop packet-size kurtosis (1.66)
Family, 5 s windows	Upload packet-size SD (0.21), download packet-size SD (0.17), packet rate (0.08)
Arch., autoencoder	Compute gap per MB (6.58), download byte rate (2.69), byte rate (2.69)
Arch., CNN	Upload packet rate (2.09), upload byte rate (2.07), byte rate (2.05)
Arch., RNN	Laptop/desktop gap ratio (13.39), Jetson/desktop throughput ratio (11.05), download byte rate (6.50)
Arch., transformer	Laptop/desktop gap ratio (0.96), download byte fraction (0.56), upload/download byte ratio (0.55)

E. Short Observation Windows

Table X reports all three levels for single windows of 0.5 to 5 s and for longer observation times of 10 to

30 s. For family identification, single-client windows reach only 0.26–0.31 per window, whereas concatenating the three clients' time-aligned windows, removing near-idle windows, and classifying with logistic regression reaches 0.41 per window at 0.5 s, rising to 0.51 at 5 s, and 0.51–0.65 after averaging window posteriors per experiment. The cross-device gain is consistent across all window sizes. The same representation identifies convolutional architectures with 0.65–0.71 per window and 0.89 after pooling at every window size, and autoencoder architectures with up to 0.78 after pooling; recurrent and transformer architectures, each represented by few models, are not reliably identifiable from short windows. Variant identification requires a complete model transfer within the window. Because model downloads are faster than uploads on the testbed, a window that begins at a download identifies the exact variant in 23% of traces at 0.5 s, rising to 69% at 5 s, and identifies it correctly whenever the download completes within the window.

TABLE X
Accuracy Versus Observation Time

	Single window				Accumulated			
Task	0.5 s	1 s	2 s	5 s	10 s	15 s	20 s	30 s
Family, single client	0.26	0.29	0.30	0.31	0.31	0.31	0.30	0.30
Family, cross-device	0.41	0.43	0.46	0.51	0.53	0.54	0.55	0.56
Arch., autoencoder	0.50	0.52	0.45	0.47	0.48	0.48	0.48	0.47
Arch., CNN	0.65	0.67	0.69	0.71	0.76	0.78	0.80	0.81
Arch., RNN	0.54	0.51	0.39	0.35	0.29	0.29	0.27	0.28
Arch., transformer	0.21	0.22	0.20	0.19	0.20	0.19	0.19	0.19
Variant†	0.23	0.32	0.39	0.69	0.80	0.82	0.86	0.90

Balanced accuracy per decision. Family and architecture: cross-device LR with near-idle windows removed, experiment-disjoint 5-fold CV grouped by model; 10–30 s average the posteriors of consecutive 5 s windows over non-overlapping spans. Chance 0.25 (family), 0.33 (autoencoder, CNN, transformer), 0.50 (RNN). †Share of traces whose variant is identified from one complete download within the observation time.

Accumulating evidence over longer observation helps where the per-window signal is consistent (Table X). Averaging the posteriors of consecutive 5 s windows raises cross-device family accuracy from 0.51 at 5 s to 0.56 at 30 s and convolutional architecture accuracy from 0.71 to 0.81, whereas the single-client baseline remains at 0.30–0.31 and autoencoder architecture does not improve. Variant identification, which depends on capturing a complete download, rises from 69% of traces at 5 s to 90% at 30 s.

F. Ablation

Table XI isolates the components of the proposed model under the same leave-one-model-out protocol with nested selection. Neither component helps in isolation: the cross-device representation with a random forest reaches 0.45, and a per-client logistic regression also reaches 0.45, both below the per-client random forest (0.58). Combined, they reach 0.74. Concatenating the three devices yields few high-dimensional examples, which tree ensembles overfit but a regularized linear model exploits. The compute-timing features contribute a further 0.15 (0.59 without them). For short windows, the cross-device representation improves both classifiers (5 s per-window accuracy from 0.31 to 0.44 with a random forest and from 0.35 to 0.49 with logistic regression), and removing near-idle windows adds 0.02 per window. The cross-device gain does not depend on a single device: with the laptop removed, combining the remaining two devices still reaches 0.71 (95% CI 0.57–0.85). As an upper bound, excluding the ten variants whose compute profiles overlap another family (for example, large ResNet and BERT models) and re-evaluating the round-level features on the remaining 19 models raises nested leave-one-model-out family accuracy to 0.85; because these exclusions were chosen from the observed errors, this figure bounds the achievable accuracy rather than estimating generalization.

TABLE XI
Ablation of the Family Model (Balanced Accuracy)

Representation	Classifier	Comp.	Bal. acc.	95% CI
Complete trace (leave-one-model-out)				
Per-client	RF	✓	0.58	0.42–0.74
Per-client	LR	✓	0.45	0.34–0.56
Cross-device	RF	✓	0.45	0.27–0.64
Cross-device	LR	–	0.59	0.45–0.75
Cross-device	LR	✓	0.74	0.58–0.88
5 s windows, per window / pooled				
Single-client	RF	–	0.31 / 0.31	
Single-client	LR	–	0.35 / 0.40	
Cross-device	RF	–	0.44 / 0.45	
Cross-device	LR	–	0.49 / 0.62	
Cross-device†	LR	–	0.51 / 0.62	

Comp.: compute-timing features. †Near-idle windows removed. Windows: experiment-disjoint 5-fold CV grouped by model.

G. Robustness to Size Padding

Padding every transfer to a size bucket defeats variant identification (Table XII): accuracy falls from 1.00 to 0.60 with 1 MB buckets and to chance (0.03) when all transfers are padded to the largest model, at 1,086% byte overhead. Family identification is unaffected, because the compute gap and cross-device timing ratios do not depend on transfer size: using these timing features alone, cross-device family identification reaches 0.62 (95% CI 0.45–0.78, $p = 0.005$), matching the same model with size features (0.60). Concealing model identity therefore requires timing defenses in addition to size padding.

TABLE XII
Variant Identification Under Size Padding

Padding	Accuracy	Byte overhead
None	1.00	0%
1 MB buckets	0.60	1.8%
Power-of-two buckets	0.45	47%
Pad to largest model	0.03	1,086%

H. Summary

The cross-device timing signature reveals the family of unseen models (0.74), also from short windows (0.51 per

5 s window, where it also identifies convolutional architectures with 0.71), and remains effective under padding (0.62). Within a family, packet-morphology features identify the architecture with a mean balanced accuracy of 0.79. Payload size reveals the exact variant (1.00, and 0.69 from a single download within a 5 s window), although padding removes this signal.

V. Countermeasures

Size padding removes the variant signal (Sec. IV-G) but at a substantial cost: padding every transfer to the largest model increases traffic by 1,086%, whereas inexpensive 1 MB buckets still allow 60% identification. Family leakage stems from timing, so effective defenses must decouple traffic timing from local computation. Candidate defenses include (i) deadline-based rounds, in which every client uploads at a fixed time after the download regardless of when training ends, hiding the compute gap at the cost of round latency; (ii) randomized upload delays, which blur the gap but must be large relative to its variation across families; (iii) cover traffic or traffic shaping that keeps the link active during local training; and (iv) relaying updates through an intermediary that batches uploads from several clients. Transport encryption and secure aggregation [17] protect update contents but leave transfer timing unchanged. Quantifying the privacy and latency trade-offs of these timing defenses is left for future work.

VI. Conclusion

This paper presented CHAI-FP, a hierarchical framework that characterizes how much AI model identity leaks from the network traffic metadata of federated learning. A passive observer that sees only packet timing, size, and direction can reconstruct the FL round structure, including the compute gap between model download and upload, and can combine the traffic of clients that train the same model on heterogeneous hardware and operating systems. On a physical testbed with Ubuntu and Windows clients, this cross-device fingerprinting identifies the family of unseen models with a balanced accuracy of 0.74 from complete traces, compared with 0.58 for per-client classification, and 0.51 from a single 5 s window, rising to 0.56 over 30 s of observation. Within a family, the architecture is identified with a mean balanced accuracy of 0.79, and the uploaded payload size reveals the exact variant in every trace and in 69% of traces from one 5 s window. Size padding removes the variant signal but not the timing signal that reveals the family, indicating that protecting model identity in FL requires timing defenses in addition to size concealment. Future work will extend the evaluation to additional architectures and repeated executions, to encrypted transports and frameworks such as Flower over gRPC, and to varying training configurations, and will quantify the privacy and latency trade-offs of timing defenses.

References

[1] Author, "Placeholder for zhou2019edge," 2026.
[2] ——, "Placeholder for mcmahan2017communication," 2026.
[3] ——, "Placeholder for sirinam2018deep," 2026.
[4] ——, "Placeholder for shuvo2025fingerprinting," 2026.
[5] ——, "Placeholder for hua2018reverse," 2026.
[6] ——, "Placeholder for yan2020cache," 2026.
[7] ——, "Placeholder for hu2020deepsniffer," 2026.
[8] ——, "Placeholder for patwari2022dnn," 2026.
[9] ——, "Placeholder for panchenko2011website," 2026.
[10] ——, "Placeholder for wang2014effective," 2026.
[11] ——, "Placeholder for hayes2016kfingerprinting," 2026.
[12] ——, "Placeholder for li2022packet," 2026.
[13] ——, "Placeholder for shuvo2026flare," 2026.
[14] ——, "Placeholder for oyaniyi2026fingerprinting," 2026.
[15] ——, "Placeholder for shuvo2026flint," 2026.
[16] ——, "Placeholder for todo-model-extraction," 2026.
[17] ——, "Placeholder for todo-secure-aggregation," 2026.